

01 — Model Training on Indian Dataset (UNet_Audio_Classifier)

Trains only `UNet_Audio_Classifier` on the Indian dataset with a clean, reproducible, and leak-free setup. Saves dataset-tagged artifacts in `models/` and `reports/`.

```
In [24]: # Minimal setup & config (fast, reproducible)
import os
from datetime import datetime

# Tiny logger
VERBOSE = os.environ.get('INDIAN_VERBOSE', '1') == '1'

def log(msg: str, level: str = 'INFO'):
    if not VERBOSE and level == 'INFO':
        return
    print(f"[{datetime.now().strftime('%H:%M:%S')}] {level}: {msg}")

# Core toggles only
INDIAN_ENABLE_XLA = os.environ.get('INDIAN_ENABLE_XLA', '0') == '1'
INDIAN_JIT_COMPILE = os.environ.get('INDIAN_JIT_COMPILE', '0') == '1'
INDIAN_MIXED_PRECISION = os.environ.get('INDIAN_MIXED_PRECISION', '1') == '1'
INDIAN_CACHE_EVAL = os.environ.get('INDIAN_CACHE_EVAL', '1') == '1'

# Device control (prefer GPU by default)
INDIAN_TRAIN_ON_GPU = os.environ.get('TRAIN_ON_GPU', '1') == '1'

# Data/time controls
INDIAN_TIME_DOWNSAMPLE = int(os.environ.get('INDIAN_TIME_DOWNSAMPLE', '2'))
INDIAN_BATCH_SIZE = int(os.environ.get('INFER_BATCH_SIZE', os.environ.get('BATCH_SIZE', 16)))

# Augmentation (slightly softened defaults)
INDIAN_SPEC_AUGMENT = os.environ.get('INDIAN_SPEC_AUGMENT', '1') == '1'
INDIAN_FREQ_MASK_PARAM = int(os.environ.get('INDIAN_FREQ_MASK_PARAM', '6'))
INDIAN_TIME_MASK_PARAM = int(os.environ.get('INDIAN_TIME_MASK_PARAM', '12'))
INDIAN_NUM_MASKS = int(os.environ.get('INDIAN_NUM_MASKS', '1'))

# Inference TTA (time shifts in frames; 0 disables TTA)
INDIAN_TTA_SHIFTS = int(os.environ.get('INDIAN_TTA_SHIFTS', '0'))

# Transfer learning toggle
INDIAN_INIT_FROM_GTZAN = os.environ.get('INDIAN_INIT_FROM_GTZAN', '1') == '1'

# Reproducibility
RANDOM_STATE = 42

log(f"CFG: XLA={INDIAN_ENABLE_XLA} | JIT={INDIAN_JIT_COMPILE} | MP={INDIAN_MIXED_PRECISION} | GPU={INDIAN_TRAIN_ON_GPU} | DS_T={INDIAN_TIME_DOWNSAMPLE} | BATCH={INDIAN_BATCH_SIZE} | TTA_SHIFT={INDIAN_TTA_SHIFTS} | INIT_FROM_GTZAN={INDIAN_INIT_FROM_GTZAN}")
```

[17:14:33] INFO: CFG: XLA=False | JIT=False | MP=True | CACHE_EVAL=True | GPU=True | DS_T=2 | BATCH=48 | AUG=True (F6,T12,N1) | TTA_SHIFT=0 | INIT_FROM_GTZAN=True

```
In [25]: # Load data and prepare environment (concise)
import os, pickle
import numpy as np, pandas as pd, tensorflow as tf, keras
from keras import layers, models, callbacks
from keras.utils import to_categorical
from pathlib import Path

# Device policy
if not INDIAN_TRAIN_ON_GPU:
    os.environ['CUDA_VISIBLE_DEVICES'] = '-1'

# Optional global XLA
try:
    if INDIAN_ENABLE_XLA:
        tf.config.optimizer.set_jit(True)
        log('XLA JIT enabled')
except Exception as e:
    log(f'XLA enable failed: {e}', level='WARN')

# Mixed precision on GPU
try:
    if INDIAN_MIXED_PRECISION and os.environ.get('CUDA_VISIBLE_DEVICES', ''):
        from keras.mixed_precision import set_global_policy
        set_global_policy('mixed_float16')
        log('Mixed precision: float16 compute / float32 vars')
except Exception as e:
    log(f'MP not enabled: {e}', level='WARN')

# Reproducibility & GPU safety
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
try:
    for gpu in tf.config.list_physical_devices('GPU'):
        tf.config.experimental.set_memory_growth(gpu, True)
except Exception as e:
    log(f'TF GPU setup: {e}', level='WARN')

# Paths
PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
PROCESSED = PROJECT_ROOT/'data'/'processed_indian'
MODELS = PROJECT_ROOT/'models'; MODELS.mkdir(exist_ok=True)
REPORTS = PROJECT_ROOT/'reports'; REPORTS.mkdir(exist_ok=True)

# Data (memmap to limit RAM)
X_train = np.load(PROCESSED/'X_train.npy', mmap_mode='r'); y_train = np.load(PROCESSED/'y_train.npy', mmap_mode='r')
X_val = np.load(PROCESSED/'X_val.npy', mmap_mode='r'); y_val = np.load(PROCESSED/'y_val.npy', mmap_mode='r')
X_test = np.load(PROCESSED/'X_test.npy', mmap_mode='r'); y_test = np.load(PROCESSED/'y_test.npy', mmap_mode='r')

# Ensure channel dim
if X_train.ndim == 3: X_train = X_train[..., None]
if X_val.ndim == 3: X_val = X_val[..., None]
if X_test.ndim == 3: X_test = X_test[..., None]
```

```

# Optional time downsample
if INDIAN_TIME_DOWNSAMPLE and INDIAN_TIME_DOWNSAMPLE > 1:
    s = int(INDIAN_TIME_DOWNSAMPLE)
    X_train, X_val, X_test = X_train[:, :, ::s, :], X_val[:, :, ::s, :], X_test[:, :, ::s, :]
    log(f'Time downsample x{s} -> train={X_train.shape}, val={X_val.shape}, test={X_test.shape}')

# Align time (crop/pad to min T)
T_min = int(min(X_train.shape[2], X_val.shape[2], X_test.shape[2]))
if not (X_train.shape[2] == X_val.shape[2] == X_test.shape[2]):
    def _pad_or_crop_time(X, T):
        cur = X.shape[2]
        if cur == T: return X
        if cur > T: return X[:, :, :T, :]
        pad = ((0,0),(0,0),(0,T-cur),(0,0))
        return np.pad(np.asarray(X), pad, mode='constant')
    X_train, X_val, X_test = (
        _pad_or_crop_time(X_train, T_min),
        _pad_or_crop_time(X_val, T_min),
        _pad_or_crop_time(X_test, T_min),
    )

with open(PROCESSED/'label_encoder.pkl','rb') as f: le = pickle.load(f)
num_classes = len(le.classes_)
y_train_cat = to_categorical(y_train, num_classes)
y_val_cat = to_categorical(y_val, num_classes)
y_test_cat = to_categorical(y_test, num_classes)

log(f'Shapes: train={X_train.shape}, val={X_val.shape}, test={X_test.shape}')

```

```

[17:14:33] WARN: Time downsample x2 -> train=(3000, 128, 64, 1), val=(1000, 128, 64, 1), test=(1000, 128, 64, 1)
[17:14:33] INFO: Shapes: train=(3000, 128, 64, 1), val=(1000, 128, 64, 1), test=(1000, 128, 64, 1) | classes=5

```

```

In [26]: # Vectorized SpecAugment and tf.data pipeline (lean)
import tensorflow as tf

def batch_spec_augment(mels, freq_mask_param=INDIAN_FREQ_MASK_PARAM, time_mask_param=INDIAN_TIME_MASK_PARAM):
    """Batch SpecAugment, operates on [B, M, T, 1]."""
    B = tf.shape(mels)[0]
    M = tf.shape(mels)[1]
    T = tf.shape(mels)[2]
    x = mels
    for _ in range(num_masks):
        if freq_mask_param > 0:
            f = tf.random.uniform([B, 1, 1, 1], 0, freq_mask_param + 1, dtype=tf.float32)
            f = tf.minimum(f, M)
            f0_max = tf.maximum(M - f, 1)
            f0 = tf.cast(tf.floor(tf.random.uniform([B, 1, 1, 1]) * tf.cast(f0_max, tf.float32)), tf.int32)
            freq_idx = tf.reshape(tf.range(M, dtype=tf.int32), [1, M, 1, 1])
            freq_mask = (freq_idx >= f0) & (freq_idx < (f0 + f))
            freq_mask = tf.broadcast_to(freq_mask, [B, M, T, 1])
            x = tf.where(freq_mask, tf.zeros([], dtype=x.dtype), x)
        if time_mask_param > 0:

```

```

        t = tf.random.uniform([B, 1, 1, 1], 0, time_mask_param + 1, dtype=t.dtype)
        t = tf.minimum(t, T)
        t0_max = tf.maximum(T - t, 1)
        t0 = tf.cast(tf.floor(tf.random.uniform([B, 1, 1, 1]) * tf.cast(t0_max, t0_max.dtype)), t0_max.dtype)
        time_idx = tf.reshape(tf.range(T, dtype=tf.int32), [1, 1, T, 1])
        time_mask = (time_idx >= t0) & (time_idx < (t0 + t))
        time_mask = tf.broadcast_to(time_mask, [B, M, T, 1])
        x = tf.where(time_mask, tf.zeros([], dtype=x.dtype), x)
    return x

def ds_with_optional_aug(X, y_cat, batch_size, training: bool):
    ds = tf.data.Dataset.from_tensor_slices((X, y_cat))
    if training:
        ds = ds.shuffle(min(10000, len(X)), seed=RANDOM_STATE, reshuffle_each_batch=True)
        # Drop remainder only on training
        ds = ds.batch(batch_size, drop_remainder=training)
        if training and INDIAN_SPEC_AUGMENT:
            def _aug(mel, y):
                mel = batch_spec_augment(mel)
                return mel, y
            ds = ds.map(_aug, num_parallel_calls=tf.data.AUTOTUNE, deterministic=True)
        if not training and INDIAN_CACHE_EVAL:
            ds = ds.cache()
    return ds.prefetch(tf.data.AUTOTUNE)

def apply_tta_time_shifts(X, shifts: int):
    """Return a list of arrays with circular time shifts for simple TTA.
    shifts=0 disables TTA.
    """
    if not shifts or shifts <= 0:
        return [X]
    T = X.shape[2]
    step = max(1, T // (shifts + 1))
    variants = [X]
    for s in range(1, shifts + 1):
        shift = (s * step) % T
        variants.append(tf.roll(X, shift=shift, axis=2).numpy())
    return variants

```

```

In [27]: # UNet architecture aligned with GTZAN tournament (model only)
from keras import layers, models, callbacks
import keras, numpy as np, pandas as pd
from sklearn.utils.class_weight import compute_class_weight
from keras.optimizers.schedules import CosineDecay
from typing import cast
from pathlib import Path

def _unet_encoder_block(input_tensor, filters, pool=True, name_prefix=""):
    x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f'{name_prefix}_conv1')(input_tensor)
    x = layers.BatchNormalization(name=f'{name_prefix}_bn1')(x)
    x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu1')(x)
    x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f'{name_prefix}_conv2')(x)
    x = layers.BatchNormalization(name=f'{name_prefix}_bn2')(x)

```

```

x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu2')(x)
skip_connection = x
if pool:
    pool_output = layers.MaxPooling2D(2, name=f'{name_prefix}_pool')(x)
    return pool_output, skip_connection
else:
    return x, skip_connection

def build_unet_audio_classifier(input_shape, num_classes):
    inputs = layers.Input(shape=input_shape)
    # Encoder path (mirrors GTZAN tournament UNet)
    p1, s1 = _unet_encoder_block(inputs, 32, name_prefix="enc1")
    p2, s2 = _unet_encoder_block(p1, 64, name_prefix="enc2")
    p3, s3 = _unet_encoder_block(p2, 128, name_prefix="enc3")
    # Bottleneck (pool=False)
    bottleneck, _ = _unet_encoder_block(p3, 256, pool=False, name_prefix="br
    # Classification head (slightly stronger regularization)
    x = layers.GlobalAveragePooling2D(name="gap")(bottleneck)
    x = layers.Dropout(float(os.environ.get('INDIAN_DROPOUT', '0.6')))(x)
    outputs = layers.Dense(num_classes, activation='softmax', dtype='float32)
    return models.Model(inputs=inputs, outputs=outputs, name='UNet_Audio_Cla

def try_load_backbone_from_gtzan(model, models_dir: Path) -> None:
    if not INDIAN_INIT_FROM_GTZAN:
        return
    for name in ['UNet_Audio_Classifier_best_WITH_AUG.keras', 'UNet_Audio_Cl
        ckpt = (models_dir / name)
        if ckpt.exists():
            try:
                src = keras.models.load_model(ckpt.as_posix(), compile=False)
                loaded = 0
                for layer in model.layers:
                    if layer.name == 'dense' or layer.name == 'logits':
                        continue
                    try:
                        src_layer = src.get_layer(layer.name)
                        if src_layer is not None and len(layer.get_weights()) > 0:
                            tgt_w = layer.get_weights(); src_w = src_layer.get_weights()
                            if all(ti.shape == si.shape for ti, si in zip(tgt_w, src_w)):
                                layer.set_weights(src_w); loaded += 1
                    except Exception:
                        pass
                print(f'Loaded {loaded} compatible layer(s) from {ckpt.name}')
                return
            except Exception as e:
                print('Backbone init failed for', ckpt.name, '->', e)
    print('No compatible GTZAN checkpoint found for backbone init.')

# Hyperparameters (tuned)
EPOCHS = int(os.environ.get('INDIAN_EPOCHS', 80)) # allow a bit more headroom
BATCH = INDIAN_BATCH_SIZE
LABEL_SMOOTH = float(os.environ.get('INDIAN_LABEL_SMOOTH', 0.01)) # slightly
LR = float(os.environ.get('INDIAN_LR', 4e-4)) # a touch lower
WEIGHT_DECAY = float(os.environ.get('INDIAN_WEIGHT_DECAY', 1e-5))

```

```

# Input shape
input_shape = tuple(int(d) for d in X_train.shape[1:])

# Data pipelines
train_ds = ds_with_optional_aug(X_train, y_train_cat, BATCH, training=True)
val_ds = ds_with_optional_aug(X_val, y_val_cat, BATCH, training=False)
test_ds = ds_with_optional_aug(X_test, y_test_cat, BATCH, training=False)

# Steps (explicit for stable progress)
train_steps = int(np.ceil(X_train.shape[0] / BATCH))
val_steps = int(np.ceil(X_val.shape[0] / BATCH))

# Learning rate schedule (incompatible with ReduceLRonPlateau)
after_total_steps = max(1, train_steps * max(1, EPOCHS))
lr_schedule = CosineDecay(initial_learning_rate=LR, decay_steps=after_total_steps)

# Build & compile model (UNet + optional GTZAN init)
model = build_unet_audio_classifier(input_shape, num_classes)
try_load_backbone_from_gtzan(model, MODELS)

opt = (
    keras.optimizers.AdamW(
        learning_rate=cast(float, lr_schedule),
        weight_decay=WEIGHT_DECAY, clipnorm=1.0
    )
    if WEIGHT_DECAY and WEIGHT_DECAY > 0 else
    keras.optimizers.Adam(learning_rate=cast(float, lr_schedule), clipnorm=1.0
)

model.compile(
    optimizer=opt,
    loss=keras.losses.CategoricalCrossentropy(label_smoothing=LABEL_SMOOTH),
    metrics=['accuracy'],
    jit_compile=INDIAN_JIT_COMPILE,
)

model.summary()

# Class weights
classes_idx = np.arange(num_classes)
class_weights_vec = compute_class_weight('balanced', classes=classes_idx, y=CLASS_WEIGHT)
CLASS_WEIGHT = {int(i): float(w) for i, w in zip(classes_idx, class_weights_vec)}

# Callbacks (Remove ReduceLRonPlateau since a schedule is used)
ckpt_path = MODELS/'UNet_Audio_Classifier_best_INDIAN.keras'
cb = [
    callbacks.ModelCheckpoint(ckpt_path, monitor='val_accuracy', save_best_only=True),
    callbacks.EarlyStopping(monitor='val_accuracy', patience=12, restore_best_weights=True)
]

# Train
history = model.fit(
    train_ds,
    validation_data=val_ds,
    steps_per_epoch=train_steps,

```

```

        validation_steps=val_steps,
        epochs=EPOCHS,
        callbacks=cb,
        class_weight=CLASS_WEIGHT,
        verbose=1,
    )

# Evaluate
# Evaluate/predict on full arrays to avoid dropping last batch
from sklearn.metrics import accuracy_score
if INDIAN_TTA_SHIFTS > 0:
    preds = []
    for Xv in apply_tta_time_shifts(X_test, INDIAN_TTA_SHIFTS):
        preds.append(model.predict(Xv, batch_size=BATCH))
    test_pred = np.mean(preds, axis=0)
else:
    test_pred = model.predict(X_test, batch_size=BATCH)
print(f'INDIAN Test Accuracy: {accuracy_score(np.argmax(y_test_cat,1), np.ar

# Save summary
pd.DataFrame([{'Model': 'UNet_Audio_Classifier', 'Dataset': 'INDIAN',
               'Best_Val_Accuracy': float(np.max(history.history.get('val_accuracy', [0
               'Test_Accuracy': float(accuracy_score(np.argmax(y_test_cat,1), np.argmax
               'Epochs_Run': int(len(history.history.get('val_accuracy', [1])))
            ]])).to_csv(REPORTS/'training_summary_INDIAN.csv', index=False)
print(f'Saved: {REPORTS/"training_summary_INDIAN.csv"}')

```

Loaded 28 compatible layer(s) from UNet_Audio_Classifier_best_WITH_AUG.keras
Model: "UNet_Audio_Classifier"

Layer (type)	Output Shape	Par
input_layer_4 (InputLayer)	(None, 128, 64, 1)	
enc1_conv1 (Conv2D)	(None, 128, 64, 32)	
enc1_bn1 (BatchNormalization)	(None, 128, 64, 32)	
enc1_prelu1 (PReLU)	(None, 128, 64, 32)	
enc1_conv2 (Conv2D)	(None, 128, 64, 32)	9
enc1_bn2 (BatchNormalization)	(None, 128, 64, 32)	
enc1_prelu2 (PReLU)	(None, 128, 64, 32)	
enc1_pool (MaxPooling2D)	(None, 64, 32, 32)	
enc2_conv1 (Conv2D)	(None, 64, 32, 64)	18
enc2_bn1 (BatchNormalization)	(None, 64, 32, 64)	
enc2_prelu1 (PReLU)	(None, 64, 32, 64)	
enc2_conv2 (Conv2D)	(None, 64, 32, 64)	36
enc2_bn2 (BatchNormalization)	(None, 64, 32, 64)	
enc2_prelu2 (PReLU)	(None, 64, 32, 64)	
enc2_pool (MaxPooling2D)	(None, 32, 16, 64)	
enc3_conv1 (Conv2D)	(None, 32, 16, 128)	73
enc3_bn1 (BatchNormalization)	(None, 32, 16, 128)	
enc3_prelu1 (PReLU)	(None, 32, 16, 128)	
enc3_conv2 (Conv2D)	(None, 32, 16, 128)	147
enc3_bn2 (BatchNormalization)	(None, 32, 16, 128)	
enc3_prelu2 (PReLU)	(None, 32, 16, 128)	
enc3_pool (MaxPooling2D)	(None, 16, 8, 128)	
bneck_conv1 (Conv2D)	(None, 16, 8, 256)	294
bneck_bn1 (BatchNormalization)	(None, 16, 8, 256)	1
bneck_prelu1 (PReLU)	(None, 16, 8, 256)	
bneck_conv2 (Conv2D)	(None, 16, 8, 256)	589
bneck_bn2 (BatchNormalization)	(None, 16, 8, 256)	1
bneck_prelu2 (PReLU)	(None, 16, 8, 256)	
gap (GlobalAveragePooling2D)	(None, 256)	

dropout_4 (Dropout)	(None , 256)	
dense_4 (Dense)	(None , 5)	1

Total params: [1,176,805](#) (4.49 MB)

Trainable params: [1,174,885](#) (4.48 MB)

Non-trainable params: [1,920](#) (7.50 KB)

Epoch 1/80

62/63  **0s** 600ms/step - accuracy: 0.3801 - loss: 1.5371

/home/alepot55/Desktop/projects/naml_project/venv/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:116: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.
self._interrupted_warning()

63/63  **44s** 655ms/step - accuracy: 0.4610 - loss: 1.3614
- val_accuracy: 0.5550 - val_loss: 1.1919
Epoch 2/80
Epoch 2/80

63/63  **40s** 635ms/step - accuracy: 0.6599 - loss: 0.8870
- val_accuracy: 0.4070 - val_loss: 1.9885
Epoch 3/80
Epoch 3/80

63/63  **41s** 658ms/step - accuracy: 0.7644 - loss: 0.6645
- val_accuracy: 0.5480 - val_loss: 1.2322
Epoch 4/80
Epoch 4/80

63/63  **42s** 659ms/step - accuracy: 0.8249 - loss: 0.5050
- val_accuracy: 0.5240 - val_loss: 1.9763
Epoch 5/80
Epoch 5/80

63/63  **44s** 696ms/step - accuracy: 0.8740 - loss: 0.3967
- val_accuracy: 0.5520 - val_loss: 1.6520
Epoch 6/80
Epoch 6/80

63/63  **42s** 665ms/step - accuracy: 0.9113 - loss: 0.3109
- val_accuracy: 0.5700 - val_loss: 1.8164
Epoch 7/80
Epoch 7/80

63/63  **43s** 679ms/step - accuracy: 0.9446 - loss: 0.2445
- val_accuracy: 0.5930 - val_loss: 1.3587
Epoch 8/80
Epoch 8/80

63/63  **42s** 668ms/step - accuracy: 0.9567 - loss: 0.2039
- val_accuracy: 0.5910 - val_loss: 1.6757
Epoch 9/80
Epoch 9/80

63/63  **44s** 700ms/step - accuracy: 0.9718 - loss: 0.1700
- val_accuracy: 0.6350 - val_loss: 1.3570
Epoch 10/80
Epoch 10/80

63/63  **44s** 693ms/step - accuracy: 0.9775 - loss: 0.1520
- val_accuracy: 0.6920 - val_loss: 1.4438
Epoch 11/80
Epoch 11/80

63/63  **43s** 679ms/step - accuracy: 0.9778 - loss: 0.1443
- val_accuracy: 0.6230 - val_loss: 2.0033
Epoch 12/80
Epoch 12/80

63/63  **42s** 670ms/step - accuracy: 0.9802 - loss: 0.1380
- val_accuracy: 0.6230 - val_loss: 1.6654
Epoch 13/80
Epoch 13/80

63/63  **42s** 672ms/step - accuracy: 0.9876 - loss: 0.1242
- val_accuracy: 0.6550 - val_loss: 1.9183
Epoch 14/80
Epoch 14/80

63/63  **43s** 682ms/step - accuracy: 0.9872 - loss: 0.1232
- val_accuracy: 0.6340 - val_loss: 1.4549
Epoch 15/80
Epoch 15/80

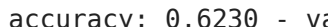
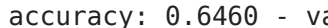
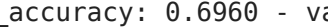
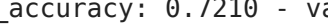
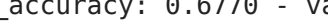
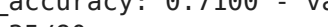
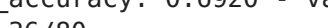
63/63  **43s** 678ms/step - accuracy: 0.9923 - loss: 0.1115
- val_accuracy: 0.5790 - val_loss: 2.1865
Epoch 16/80
Epoch 16/80

63/63  **43s** 679ms/step - accuracy: 0.9929 - loss: 0.1056
- val_accuracy: 0.7050 - val_loss: 1.1152
Epoch 17/80
Epoch 17/80

63/63  **42s** 672ms/step - accuracy: 0.9889 - loss: 0.1095
- val_accuracy: 0.6930 - val_loss: 1.5677
Epoch 18/80
Epoch 18/80

63/63  **44s** 704ms/step - accuracy: 0.9926 - loss: 0.1069
- val_accuracy: 0.5040 - val_loss: 3.0422
Epoch 19/80
Epoch 19/80

63/63  **43s** 681ms/step - accuracy: 0.9950 - loss: 0.0989

- val_accuracy: 0.6800 - val_loss: 1.2704
Epoch 20/80
63/63  **46s** 738ms/step - accuracy: 0.9973 - loss: 0.0944
- val_accuracy: 0.7110 - val_loss: 1.2221
Epoch 21/80
63/63  **45s** 713ms/step - accuracy: 0.9983 - loss: 0.0934
- val_accuracy: 0.6520 - val_loss: 1.7608
Epoch 22/80
63/63  **45s** 709ms/step - accuracy: 0.9943 - loss: 0.1019
- val_accuracy: 0.6550 - val_loss: 1.7086
Epoch 23/80
63/63  **45s** 714ms/step - accuracy: 0.9963 - loss: 0.0969
- val_accuracy: 0.5580 - val_loss: 2.7032
Epoch 24/80
63/63  **58s** 930ms/step - accuracy: 0.9946 - loss: 0.0973
- val_accuracy: 0.6180 - val_loss: 1.7801
Epoch 25/80
63/63  **44s** 695ms/step - accuracy: 0.9956 - loss: 0.0940
- val_accuracy: 0.6720 - val_loss: 1.5320
Epoch 26/80
63/63  **47s** 740ms/step - accuracy: 0.9966 - loss: 0.0928
- val_accuracy: 0.6790 - val_loss: 1.1750
Epoch 27/80
63/63  **44s** 696ms/step - accuracy: 0.9970 - loss: 0.0922
- val_accuracy: 0.6300 - val_loss: 1.6120
Epoch 28/80
63/63  **43s** 688ms/step - accuracy: 0.9983 - loss: 0.0871
- val_accuracy: 0.4840 - val_loss: 3.4775
Epoch 29/80
63/63  **44s** 695ms/step - accuracy: 0.9960 - loss: 0.0907
- val_accuracy: 0.6230 - val_loss: 1.6553
Epoch 30/80
63/63  **43s** 682ms/step - accuracy: 0.9976 - loss: 0.0901
- val_accuracy: 0.6460 - val_loss: 1.8180
Epoch 31/80
63/63  **43s** 677ms/step - accuracy: 0.9966 - loss: 0.0894
- val_accuracy: 0.6960 - val_loss: 1.4682
Epoch 32/80
63/63  **43s** 684ms/step - accuracy: 0.9970 - loss: 0.0880
- val_accuracy: 0.7210 - val_loss: 1.2234
Epoch 33/80
63/63  **43s** 685ms/step - accuracy: 0.9990 - loss: 0.0858
- val_accuracy: 0.6770 - val_loss: 1.4534
Epoch 34/80
63/63  **43s** 682ms/step - accuracy: 0.9970 - loss: 0.0895
- val_accuracy: 0.7100 - val_loss: 1.1826
Epoch 35/80
63/63  **43s** 684ms/step - accuracy: 0.9990 - loss: 0.0837
- val_accuracy: 0.6920 - val_loss: 1.0789
Epoch 36/80
63/63  **43s** 682ms/step - accuracy: 0.9997 - loss: 0.0810
- val_accuracy: 0.6540 - val_loss: 1.4806
Epoch 37/80
63/63  **42s** 671ms/step - accuracy: 0.9993 - loss: 0.0827
- val_accuracy: 0.7200 - val_loss: 1.0766
Epoch 38/80

63/63 ————— 43s 682ms/step - accuracy: 0.9997 - loss: 0.0801
- val_accuracy: 0.7100 - val_loss: 1.2350
Epoch 39/80

63/63 ————— 42s 670ms/step - accuracy: 0.9990 - loss: 0.0813
- val_accuracy: 0.7070 - val_loss: 1.2065
Epoch 40/80

63/63 ————— 42s 671ms/step - accuracy: 0.9980 - loss: 0.0820
- val_accuracy: 0.7580 - val_loss: 0.9144
Epoch 41/80

63/63 ————— 43s 679ms/step - accuracy: 0.9993 - loss: 0.0810
- val_accuracy: 0.7010 - val_loss: 1.2510
Epoch 42/80

63/63 ————— 43s 676ms/step - accuracy: 0.9990 - loss: 0.0808
- val_accuracy: 0.7170 - val_loss: 1.0692
Epoch 43/80

63/63 ————— 42s 673ms/step - accuracy: 0.9987 - loss: 0.0803
- val_accuracy: 0.7390 - val_loss: 0.9548
Epoch 44/80

63/63 ————— 43s 676ms/step - accuracy: 0.9993 - loss: 0.0812
- val_accuracy: 0.7150 - val_loss: 1.2575
Epoch 45/80

63/63 ————— 43s 682ms/step - accuracy: 0.9990 - loss: 0.0806
- val_accuracy: 0.7420 - val_loss: 1.0425
Epoch 46/80

63/63 ————— 43s 683ms/step - accuracy: 1.0000 - loss: 0.0762
- val_accuracy: 0.7510 - val_loss: 0.9276
Epoch 47/80

63/63 ————— 43s 683ms/step - accuracy: 0.9987 - loss: 0.0778
- val_accuracy: 0.7230 - val_loss: 1.0634
Epoch 48/80

63/63 ————— 43s 684ms/step - accuracy: 0.9983 - loss: 0.0790
- val_accuracy: 0.7040 - val_loss: 1.2585
Epoch 49/80

63/63 ————— 43s 685ms/step - accuracy: 0.9993 - loss: 0.0781
- val_accuracy: 0.7620 - val_loss: 0.8576
Epoch 50/80

63/63 ————— 43s 683ms/step - accuracy: 0.9990 - loss: 0.0775
- val_accuracy: 0.7400 - val_loss: 0.9281
Epoch 51/80

63/63 ————— 43s 688ms/step - accuracy: 0.9993 - loss: 0.0764
- val_accuracy: 0.7100 - val_loss: 1.1360
Epoch 52/80

63/63 ————— 43s 688ms/step - accuracy: 0.9997 - loss: 0.0759
- val_accuracy: 0.7350 - val_loss: 0.9299
Epoch 53/80

63/63 ————— 43s 679ms/step - accuracy: 0.9993 - loss: 0.0767
- val_accuracy: 0.7320 - val_loss: 0.9281
Epoch 54/80

63/63 ————— 43s 676ms/step - accuracy: 0.9993 - loss: 0.0767
- val_accuracy: 0.7370 - val_loss: 0.9339
Epoch 55/80

63/63 ————— 42s 673ms/step - accuracy: 0.9980 - loss: 0.0775
- val_accuracy: 0.7180 - val_loss: 0.9457
Epoch 56/80

63/63 ————— 43s 683ms/step - accuracy: 0.9997 - loss: 0.0765
- val_accuracy: 0.7440 - val_loss: 0.8729

Epoch 57/80
63/63  **42s** 665ms/step - accuracy: 1.0000 - loss: 0.0751
 - val_accuracy: 0.7540 - val_loss: 0.8862
 Epoch 58/80
63/63  **42s** 664ms/step - accuracy: 0.9993 - loss: 0.0762
 - val_accuracy: 0.7560 - val_loss: 0.8680
 Epoch 59/80
63/63  **43s** 677ms/step - accuracy: 0.9993 - loss: 0.0756
 - val_accuracy: 0.7520 - val_loss: 0.8459
 Epoch 60/80
63/63  **43s** 682ms/step - accuracy: 0.9997 - loss: 0.0742
 - val_accuracy: 0.7250 - val_loss: 0.9193
 Epoch 61/80
63/63  **42s** 669ms/step - accuracy: 0.9990 - loss: 0.0763
 - val_accuracy: 0.7590 - val_loss: 0.8247
21/21  **5s** 208ms/step
 INDIAN Test Accuracy: 0.7220
 Saved: /home/alepot55/Desktop/projects/naml_project/reports/training_summary_INDIAN.csv

```
In [28]: # Metrics: classification report and confusion matrix
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import seaborn as sns, matplotlib.pyplot as plt, numpy as np, pandas as pd

# Predict (full arrays)
y_val_pred = model.predict(X_val, batch_size=BATCH)
y_test_pred = model.predict(X_test, batch_size=BATCH)

# VAL report/CM
val_true_idx = np.argmax(y_val_cat, axis=1)
val_pred_idx = np.argmax(y_val_pred, axis=1)
val_report = classification_report(val_true_idx, val_pred_idx, target_names=le.classes_)
with open(REPORTS/'classification_report_UNet_Audio_Classifier_INDIAN_VAL.txt', 'w') as f:
    f.write(str(val_report))
cm_val = confusion_matrix(val_true_idx, val_pred_idx)
plt.figure(figsize=(8,6))
sns.heatmap(cm_val, annot=True, fmt='d', cmap='Blues', xticklabels=list(le.classes_), yticklabels=list(le.classes_))
plt.xlabel('Predicted'); plt.ylabel('True'); plt.title('Confusion Matrix - Validation')
plt.tight_layout(); plt.savefig(REPORTS/'confusion_matrix_UNet_Audio_Classifier_INDIAN_VAL.png')

# TEST report/CM
test_true_idx = np.argmax(y_test_cat, axis=1)
test_pred_idx = np.argmax(y_test_pred, axis=1)
report = classification_report(test_true_idx, test_pred_idx, target_names=le.classes_)
with open(REPORTS/'classification_report_UNet_Audio_Classifier_INDIAN.txt', 'w') as f:
    f.write(str(report))
cm = confusion_matrix(test_true_idx, test_pred_idx)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=list(le.classes_), yticklabels=list(le.classes_))
plt.xlabel('Predicted'); plt.ylabel('True'); plt.title('Confusion Matrix - Test')
plt.tight_layout(); plt.savefig(REPORTS/'confusion_matrix_UNet_Audio_Classifier_INDIAN.png')

print('Saved VAL/TEST reports and confusion matrices to', REPORTS)
```

21/21  3s 163ms/step

21/21  3s 162ms/step

Saved VAL/TEST reports and confusion matrices to /home/alepot55/Desktop/projects/naml_project/reports